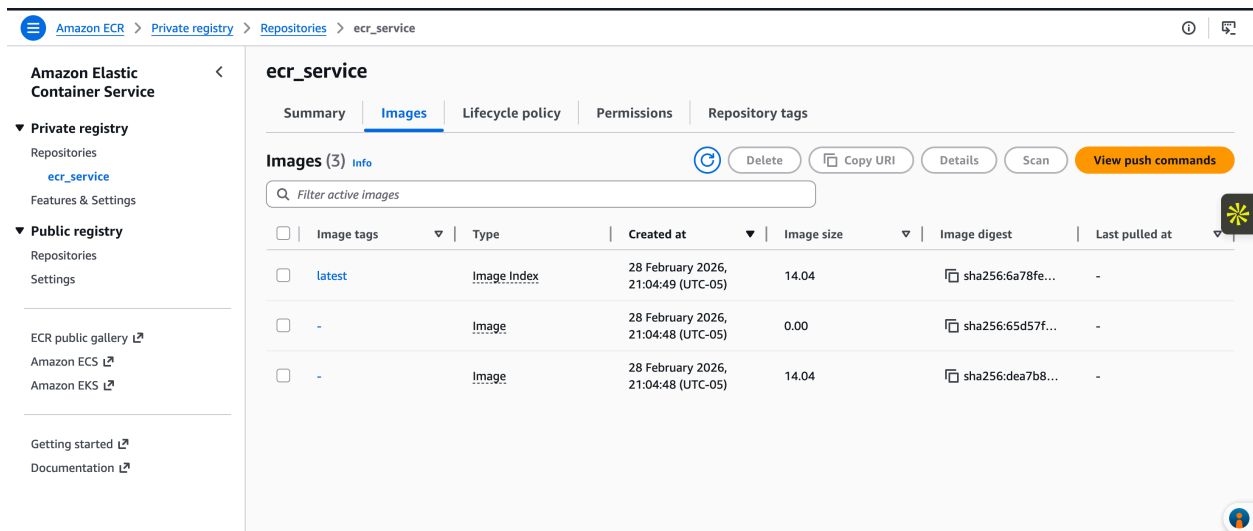


Midterm Mastery 2

This experiment demonstrates cascading failure and recovery in a microservices architecture. The search service depends on a flaky recommendation service that randomly sleeps 2-10 seconds and fails 30% of the time. Phase 1 shows what happens when slow downstream calls block goroutines and starve the entire service. Phase 2 introduces two resiliency patterns - a 500ms fail-fast timeout and a circuit breaker to show how the same broken downstream can be contained without fixing it.

ECR repo:

The Docker image containing the updated search service with `recommend.go`, circuit breaker logic, and the `/debug/circuit` endpoint is pushed to the ECR repository.



ECS:

The ECS cluster runs the service on AWS Fargate with 2 tasks behind an Application Load Balancer. Tasks are configured with 256 CPU units and 512MB memory.

Amazon Elastic Container Service > Clusters > CS6650L2-cluster > Tasks

Express Mode
Clusters
Namespaces
Task definitions
Account settings

Amazon ECR
Repositories

AWS Batch

Online learning workshop
Documentation
Discover products
Subscriptions

Tell us what you think

CS6650L2-cluster

Last updated 28 February 2026, 21:18 (UTC-5:00)

Actions Create with Express Mode

Cluster overview

ARN: arn:aws:ecs:us-east-1:094522691:730:cluster/CS6650L2-cluster

Status: Active

CloudWatch monitoring: Default

Registered container instances: -

Services

Draining: -

Active: 1

Tasks

Pending: 1

Running: 2

Services | **Tasks** | Infrastructure | Metrics | Scheduled tasks | Configuration | Event history | Tags

Tasks (12)

Last updated 28 February 2026, 21:17 (UTC-5:00)

Manage tags Stop Run new task

Filter tasks by property or value

Filter desired status: Any desired status

Filter launch type: Any launch type

☐	Task	Last status	Desired status	Task definition	Health status	Created at
☐	429c91d726ab4cd2b9f33...	Running	Running	CS6650L2-task:4	Unknown	12 minutes ago
☐	d0cd180eb4ed4922a414...	Running	Running	CS6650L2-task:4	Unknown	13 minutes ago

Testing health of ALB and response testing:

The /health endpoint confirms the ALB is routing traffic correctly. The /debug/circuit endpoint shows circuit_breaker_state: closed and protection_enabled: false, confirming Phase 1 is running with no protection — goroutines will hang indefinitely on slow downstream calls.

```
(base) srinansiramesh@Srimansis-Laptop Midterm_mystery_2 % curl http://CS6650L2-alb-757775397.us-east-1.elb.amazonaws.com/health
{"status":"healthy"}
```

Checking search endpoint using ALB:

```
(base) srinansiramesh@Srimansis-Laptop Midterm_mystery_2 % curl "http://CS6650L2-alb-757775397.us-east-1.elb.amazonaws.com/products/search?q=alpha"
{"products":[{"id":8,"name":"Product Alpha 8","category":"Health","description":"Affordable and reliable everyday item","brand":"Alpha"},{"id":16,"name":"Product Alpha 16","category":"Clothing","description":"Best seller in its category","brand":"Alpha"},{"id":24,"name":"Product Alpha 24","category":"Sports","description":"Top rated by customers worldwide","brand":"Alpha"},{"id":32,"name":"Product Alpha 32","category":"Home","description":"Premium grade with warranty included","brand":"Alpha"},{"id":40,"name":"Product Alpha 40","category":"Electronics","description":"High quality product with excellent features","brand":"Alpha"},{"id":48,"name":"Product Alpha 48","category":"Health","description":"Affordable and reliable everyday item","brand":"Alpha"},{"id":56,"name":"Product Alpha 56","category":"Clothing","description":"Best seller in its category","brand":"Alpha"},{"id":64,"name":"Product Alpha 64","category":"Sports","description":"Top rated by customers worldwide","brand":"Alpha"},{"id":72,"name":"Product Alpha 72","category":"Home","description":"Premium grade with warranty included","brand":"Alpha"},{"id":80,"name":"Product Alpha 80","category":"Electronics","description":"High quality product with excellent features","brand":"Alpha"},{"id":88,"name":"Product Alpha 88","category":"Health","description":"Affordable and reliable everyday item","brand":"Alpha"},{"id":96,"name":"Product Alpha 96","category":"Clothing","description":"Best seller in its category","brand":"Alpha"}],"total_found":12,"search_time":0.853866651,"recommendations":[{"id":1,"name":"Product Beta 1","category":"Electronics","description":"High quality product with excellent features","brand":"Beta"},{"id":2,"name":"Product Beta 2","category":"Electronics","description":"High quality product with excellent features","brand":"Beta"}]}
```

Checking /circuit endpoint to see if protection enabled is FALSE

```
(base) srinansiramesh@Srimansis-Laptop Midterm_mystery_2 % curl http://CS6650L2-alb-757775397.us-east-1.elb.amazonaws.com/debug/circuit
{"circuit_breaker_state":"closed","protection_enabled":false}
```

Load Testing:

1.5 users and 5 spawn rate

LOCUST

Host

http://CS6650L2-alb-757775397.us-east-1.elb.amaz...

Status

STOPPED

RPS

1.1

Failures

0%

NEW

RESET

STATISTICS

CHARTS

FAILURES

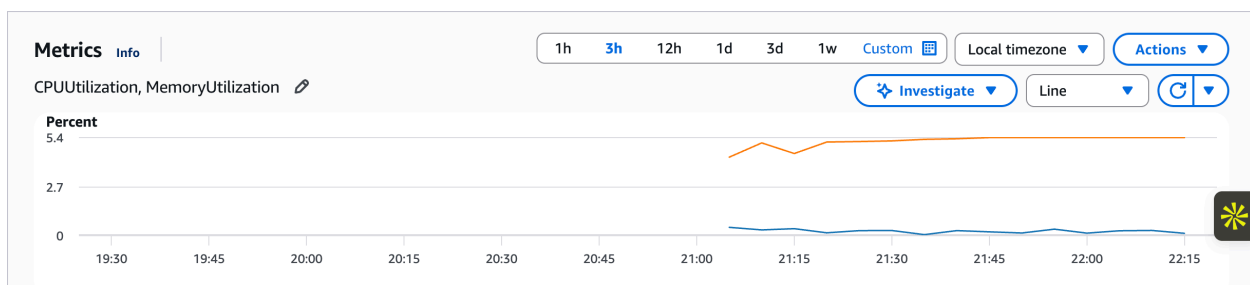
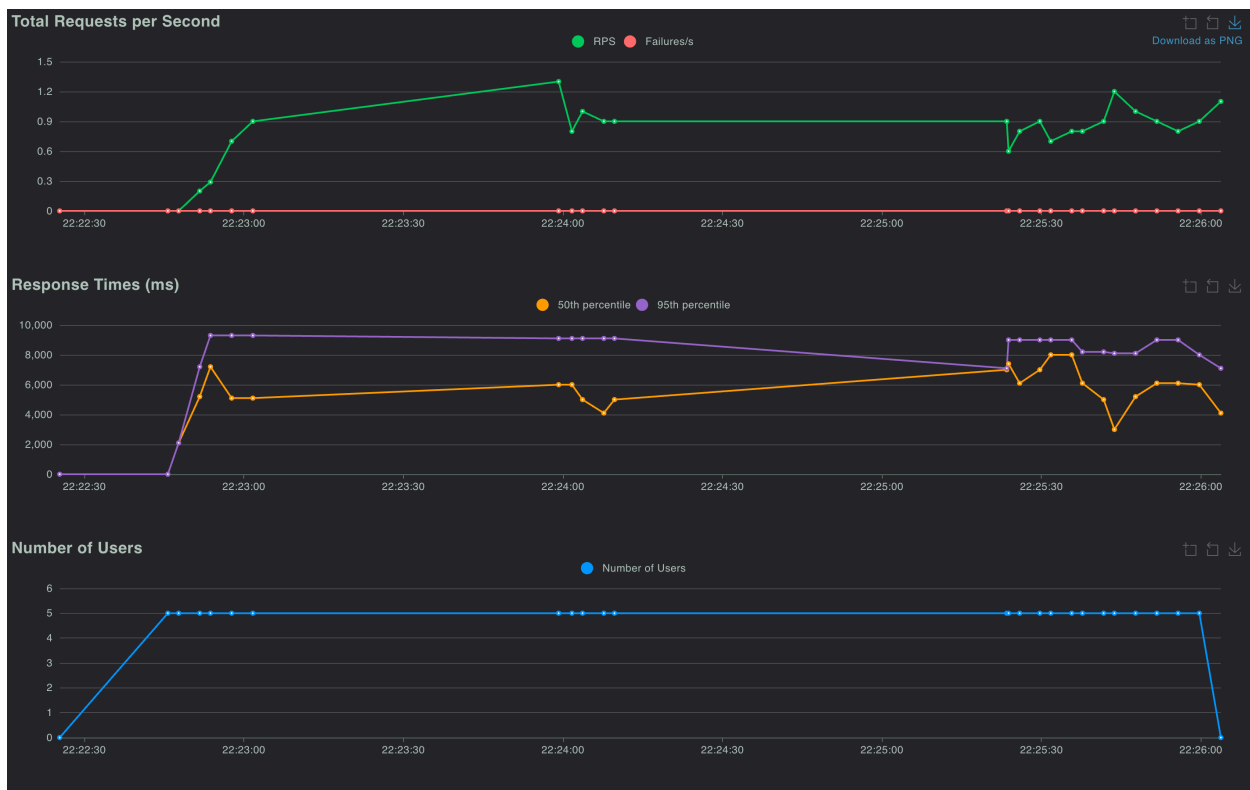
EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

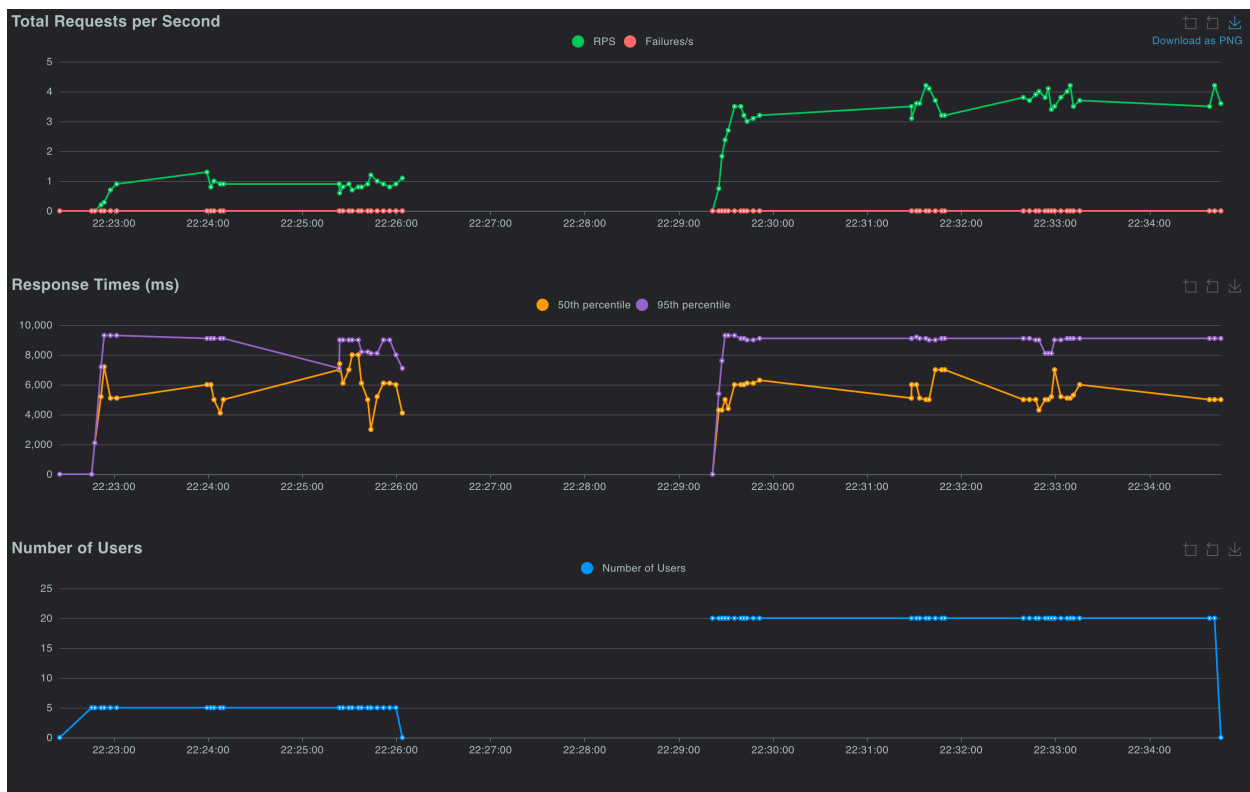
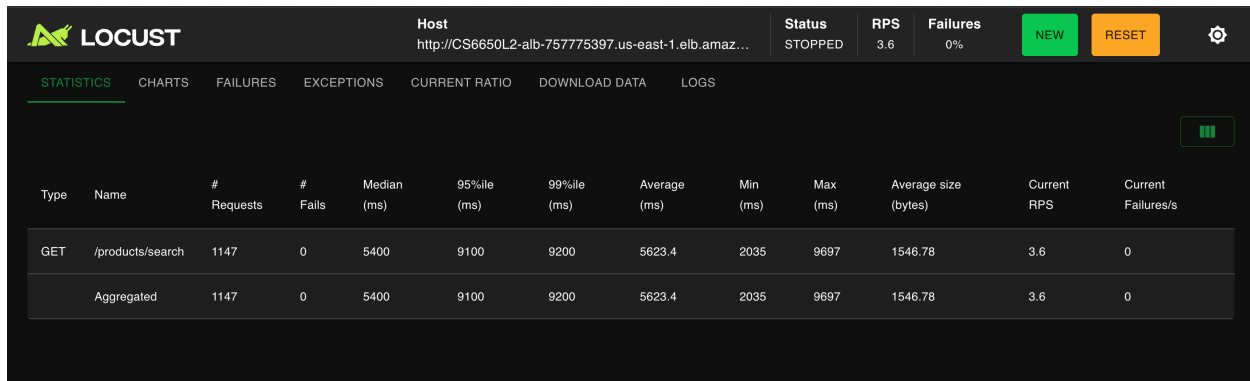
LOGS

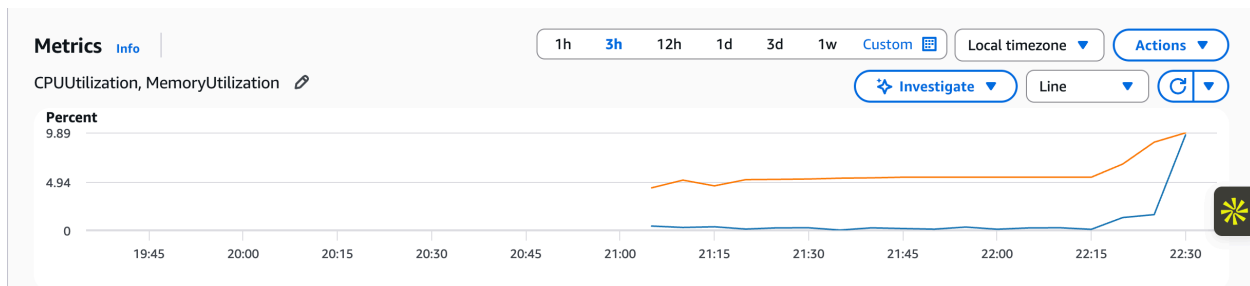
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	183	0	5100	9000	9100	5314.93	2037	9345	1574.85	1.1	0
	Aggregated	183	0	5100	9000	9100	5314.93	2037	9345	1574.85	1.1	0



RPS of 1.1 despite 5 concurrent users which means each request takes 5+ seconds waiting for the flaky recommend service. Memory (5.4%) already exceeds CPU (0.5%), this is the signal that the goroutines are blocked by I/O

2. 20 users and 20 spawn rate





4x more users yields only 3.6x RPS. Response times remain stuck at 5-9 seconds regardless of user count signifying that the bottleneck is the downstream service, not compute. Memory doubles to 9.9% as goroutines accumulate.

3. 50 users and 50 spawn rate

 LOCUST

Host

http://CS6650L2-alb-757775397.us-east-1.elb.amaz...

Status

STOPPED

RPS

8.6

Failures

0%

NEW

RESET



STATISTICS

CHARTS

FAILURES

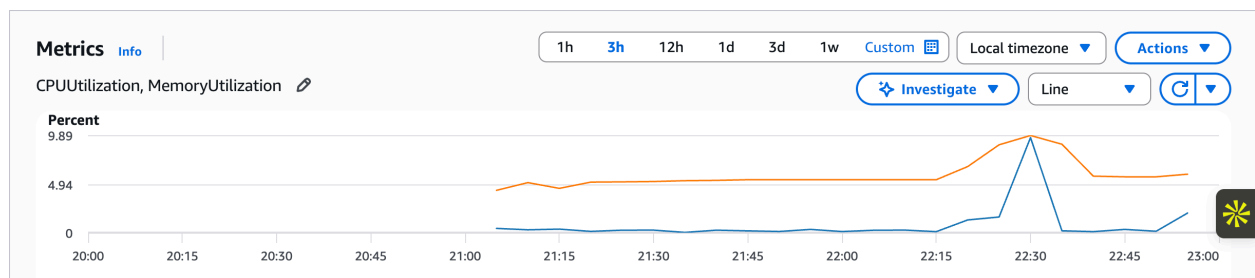
EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	2998	0	5200	9100	9200	5513	2032	11051	1548.2	8.6	0
	Aggregated	2998	0	5200	9100	9200	5513	2032	11051	1548.2	8.6	0



RPS scales linearly with users (~0.17 RPS per user) but response times do not improve. The service is in a permanent slow crawl not failing outright, but effectively useless.

Fix - circuit breaker and fail fast

PROTECTION_ENABLED is set to true via a new ECS task definition revision, triggering a rolling deployment. The /debug/circuit endpoint now shows protection_enabled: true. Two patterns are now active: a 500ms context timeout that cuts off slow downstream calls immediately, and a circuit breaker that opens after 5 consecutive failures and stops calling the downstream entirely for 30 seconds.

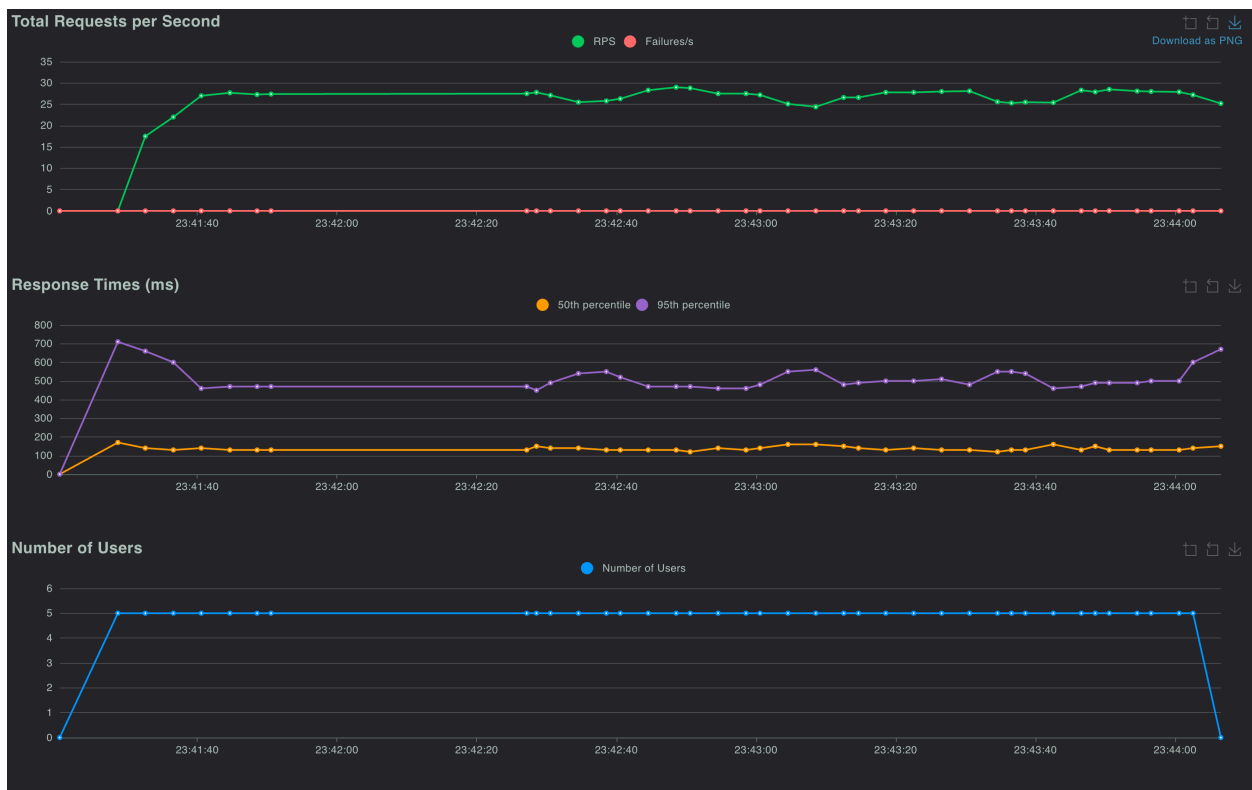
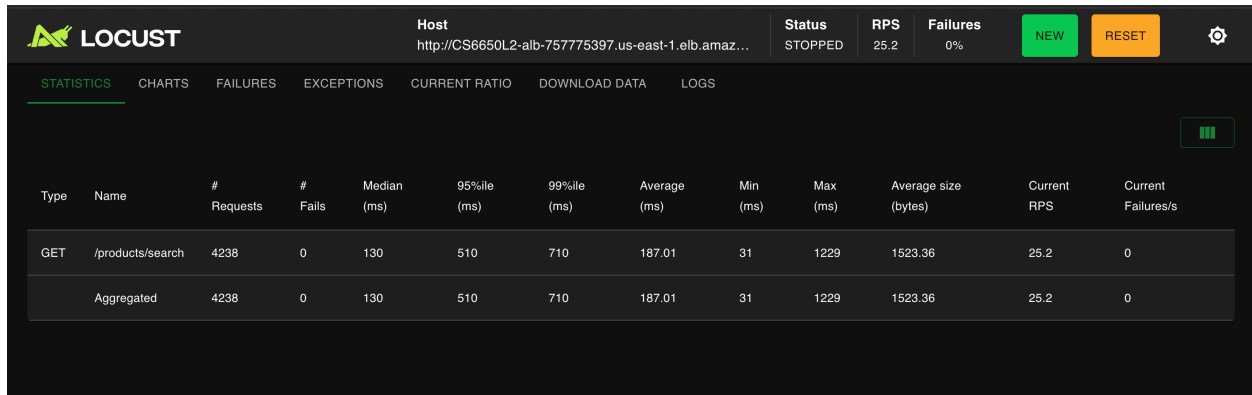
```

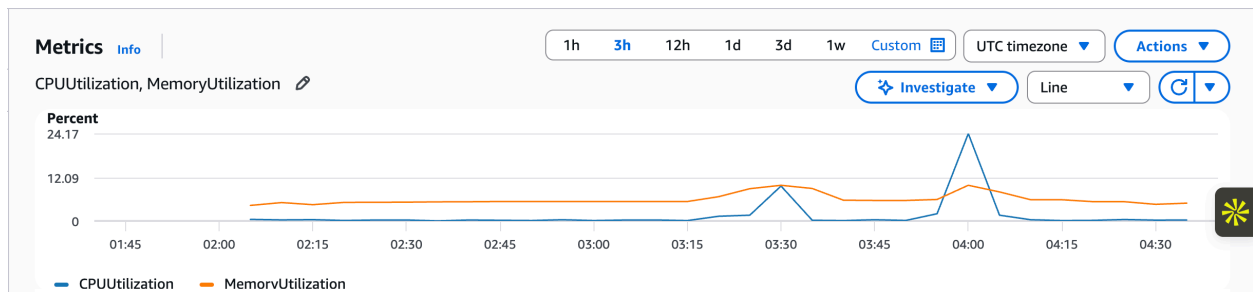
{"circuit_breaker_state":"closed","protection_enabled":false}
● (base) srinansiramesh@Srimansis-Laptop Midterm_mystery_2 % curl http://CS6650L2-alb-757775397.us-east-1.elb.amazonaws.com/debug/circuit
{"circuit_breaker_state":"closed","protection_enabled":true}
○ (base) srinansiramesh@Srimansis-Laptop Midterm_mystery_2 %

```

Load testing

1.5 users and 5 spawn rate





RPS jumps to 25.2 which is a 25x improvement over Phase 1. Median drops from 5,100ms to 130ms. CPU (24%) now exceeds memory (8%), the opposite of Phase 1. The service is doing real compute work again instead of blocking on I/O. The 95th percentile at 510ms reflects the 500ms timeout kicking in.

2. 20 users and 20 spawn rate

 LOCUST

Host

http://CS6650L2-alb-757775397.us-east-1.elb.amaz...

Status

STOPPED

RPS

25.7

Failures

0%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

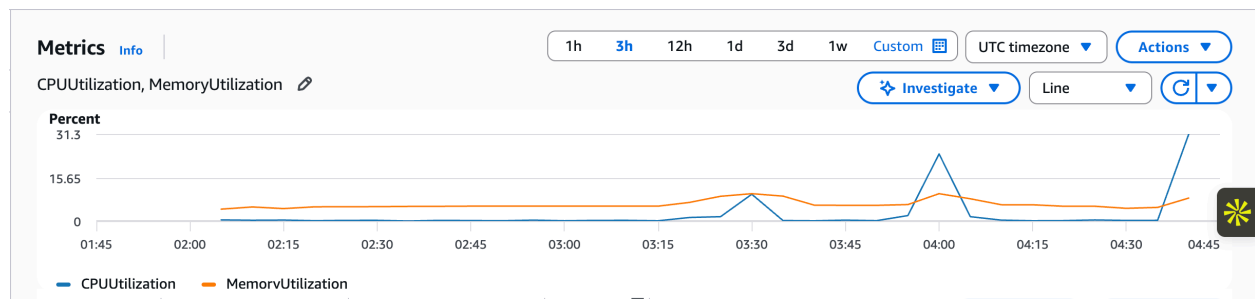
LOGS

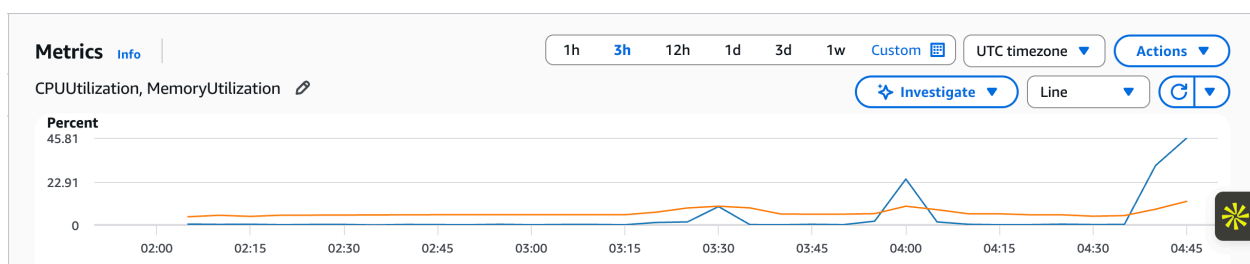
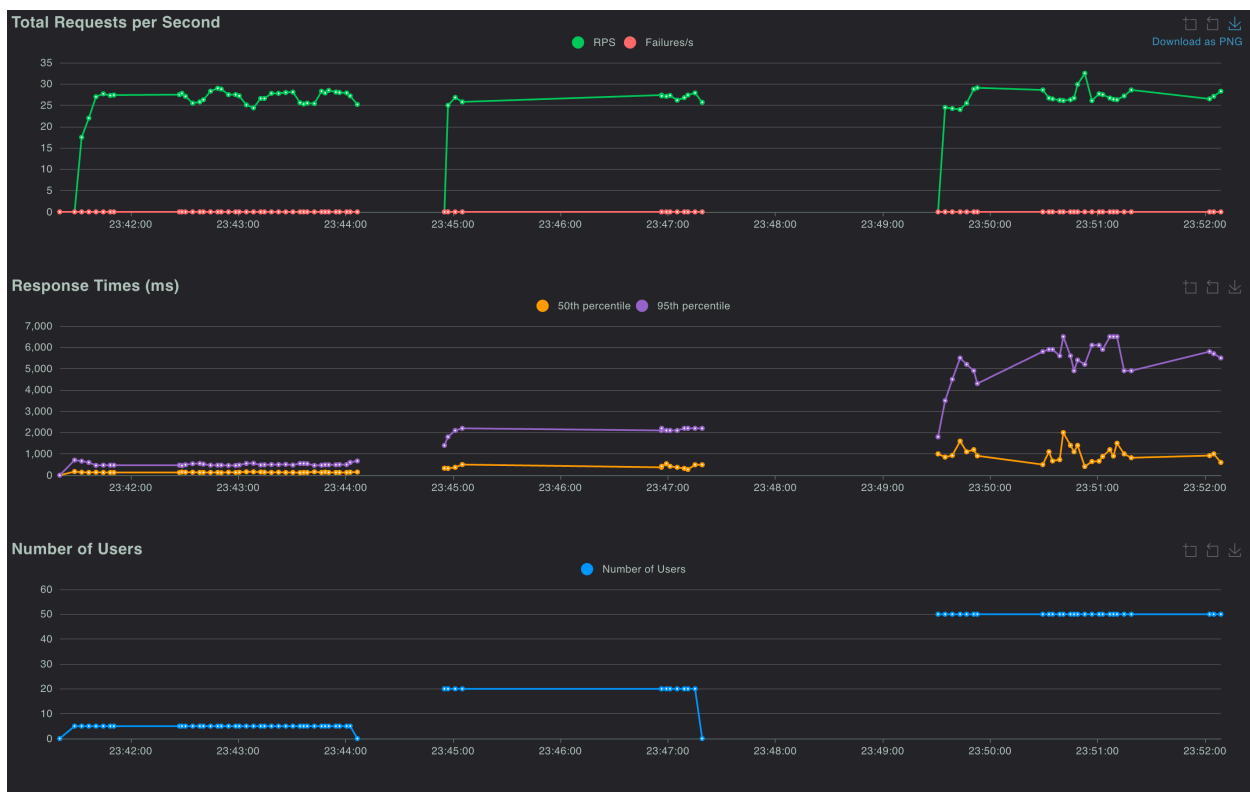
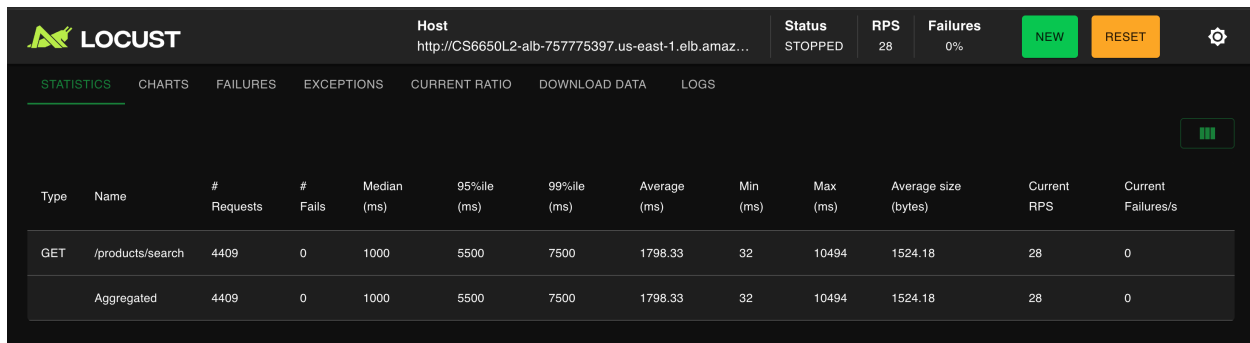
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	3888	0	430	2100	2700	731.3	30	4192	1527.39	25.7	0
	Aggregated	3888	0	430	2100	2700	731.3	30	4192	1527.39	25.7	0



RPS holds at 25.7 which is nearly identical to 5 users, showing the circuit breaker is now the throughput ceiling, not compute. The 95th percentile rises to 2,100ms as the circuit breaker cycles between OPEN and HALF_OPEN states under higher concurrency.

3. 50 users and 50 spawn rate





At 50 users, CPU climbs to 45.81%, the service is now CPU-saturated from SHA-256 computation. The bottleneck has shifted from I/O blocking back to compute, meaning the fix for Phase 2 at high load is horizontal scaling.

Comparison of results

Phase 1 — No Protection (Crash)					
Users	RPS	Median	95th %ile	CPU	Memory
5	~1.0	5,100ms	9,000ms	~0.5%	~5.4%
20	~3.6	5,400ms	9,100ms	~2%	~9.9%
50	~8.6	5,200ms	9,100ms	~2%	~5%
Phase 2 — With Protection (Recovery)					
Users	RPS	Median	95th %ile	CPU	Memory
5	~27	130ms	510ms	~5%	~8%
20	~25	430ms	2,100ms	~22%	~8%
50	~28	1,000ms	5,500ms	~45%	~10%

The contrast is stark: enabling two code-level resiliency patterns improved throughput by 7-27x and reduced median response time by 12-39x, with the downstream service still broken throughout. Phase 1 shows Memory > CPU (I/O bound, goroutines blocked). Phase 2 shows CPU > Memory (compute bound, real work being done). Importantly, horizontal scaling cannot fix Phase 1 as adding more tasks just adds more blocked goroutines. The circuit breaker and timeout must come first.